

Parallel Computing

Personal Notes

Mihir Rao

Contents

1	Why Parallelism? Why Efficiency?	2
1.1	What is a Parallel Computer?	2
1.2	Three Course Themes	3
1.3	Why Parallelism Now?	4
1.4	The Parallel Hardware Landscape	6
1.5	Memory and the Cost of Data Movement	7
1.6	Summary	10

1 Why Parallelism? Why Efficiency?

The course has two interleaved aims: writing programs that run efficiently on modern parallel hardware, and understanding the hardware well enough to know why some programs scale and others do not. This first lecture is mostly motivation. We give a working definition of a parallel computer, introduce speedup, and explain why every modern processor — from a phone SoC to a data-center GPU — is parallel.

1.1 What is a Parallel Computer?

Definition 1.1 (Parallel computer). A *parallel computer* is a collection of processing elements that cooperate to solve problems quickly.

Two words in this definition do real work. “Cooperate” implies *communication* and *synchronization* between processing elements: they share data and coordinate progress. “Quickly” is the reason we bother in the first place — we are using more than one processor because we want the answer sooner, or because a single processor cannot keep up with the problem at all.

***Note: Two intertwined goals.** Throughout the course we will care simultaneously about performance (getting the answer fast) and efficiency (using the hardware’s resources well). They are related but not the same: a program can run faster on a parallel machine and still be terrible at using its execution units.*

1.1.1 Speedup

The first quantitative tool is speedup.

Definition 1.2 (Speedup). For a fixed problem, the *speedup* of a parallel implementation using P processors is

$$S(P) = \frac{T_1}{T_P},$$

where T_1 is the execution time on one processor and T_P is the execution time on P processors.

Ideally $S(P) = P$ (*linear speedup*). In practice, $S(P) < P$, often by a wide margin, for reasons that we will study repeatedly:

- *Communication.* Processors must exchange data. The cost of moving that data eventually dominates computation, especially as P grows.
- *Load imbalance.* If some processors finish their assigned work early and sit idle while others continue, the parallel time is set by the slowest, not the average.

- *Synchronization and dependencies.* Some operations must wait for others to finish. The longest chain of dependent work bounds the achievable speedup regardless of P .

Example: Three classroom demos. The lecture's three live demos make these forces concrete. In a small partial-sum demo with a handful of "processors" (students), communication cost was the binding constraint: putting the students closer together (or letting them shout) shrank the constant in front of P and improved speedup. With four "processors" the binding constraint shifted to load imbalance — some students ran out of work and went idle while others were still summing. With many "processors," communication once again dominated, and adding more people made the program *slower*, not faster.

1.2 Three Course Themes

The class is organised around three themes, each of which corresponds roughly to a perspective on the same artifact.

1.2.1 Theme 1 — Designing parallel programs that scale

Parallel thinking consists of three concrete tasks:

1. *Decomposing* the work into pieces that can safely be performed in parallel.
2. *Assigning* those pieces to processors.
3. *Managing* communication and synchronization between processors so that they do not negate the speedup we hoped to win.

We will study abstractions and mechanisms (ISPC, pthreads, OpenMP, CUDA, MPI) that let us express these decisions cleanly in code.

1.2.2 Theme 2 — How parallel hardware actually works

A parallel program lives on top of an implementation: caches, memory hierarchies, vector units, multiple cores, multiple sockets, GPUs, accelerators. Performance characteristics of those implementations and the design tradeoffs behind them (performance vs. convenience vs. cost in silicon area, power, and chip complexity) are an explicit subject of the course, not a hidden detail.

Note: Why hardware. *The characteristics of the machine matter. Recall the demos: changing how processors communicated changed the speedup. Caring about performance forces us to care about the hardware.*

1.2.3 Theme 3 — Thinking about efficiency

Fast is not the same as efficient. A program that runs 2× faster on a machine with ten processors has captured only 20% of the available parallelism; the other eight processors are wasted. From the programmer's perspective, efficiency means making use of provided machine capabilities; from the hardware designer's perspective, it means choosing capabilities that pay off in performance per unit of silicon area or per watt.

1.3 Why Parallelism Now?

For decades, single-threaded CPU performance roughly doubled every 18 months, and the rational software response was to wait: code that was too slow this year would be fast enough next year. Two engines drove that improvement:

1. exploiting *instruction-level parallelism* (ILP) via superscalar, out-of-order execution; and
2. increasing the CPU clock frequency.

Roughly fifteen years ago, both engines stalled. Understanding why is the reason this course exists.

1.3.1 Programs as instruction streams

A program, from a processor's perspective, is a list of instructions. The small C program

```
1 int main(int argc, char** argv) {  
2     int x = 1;  
3     for (int i = 0; i < 10; i++) {  
4         x = x + x;  
5     }  
6     printf("%d\n", x);  
7     return 0;  
8 }
```

compiles into a sequence of machine instructions of the form

```
1 movl $1, -20(%rbp)  
2 movl $0, -24(%rbp)  
3 cmpl $10, -24(%rbp)  
4 jge end  
5 movl -20(%rbp), %eax  
6 addl -20(%rbp), %eax
```

A processor executes these instructions one at a time: *fetch and decode* the next instruction, *read* its inputs from registers, *execute* the operation in an arithmetic logic unit (ALU), *write back* the result. On the simplest in-order processor each instruction takes one clock cycle, and the program runs in (number of instructions) $\times$ (cycle time) seconds.

Note: Computer state. A program's state is just the values stored in the processor's registers and in memory. An instruction's job is to transform that state in some small, well-defined way.

1.3.2 Instruction-level parallelism

Consider the trivial computation

$$a = x^2 + y^2 + z^2,$$

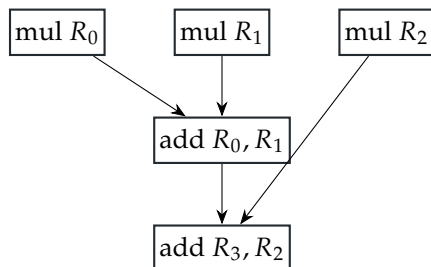
which compiles to (with $R_0 = x$, $R_1 = y$, $R_2 = z$):

```

1 1  mul R0, R0, R0
2 2  mul R1, R1, R1
3 3  mul R2, R2, R2
4 4  add R0, R0, R1
5 5  add R3, R0, R2

```

A naive in-order processor takes five cycles. But instructions 1, 2, and 3 are mutually independent — their inputs are disjoint and their outputs are disjoint — so a processor with two ALUs can issue any two of them in the same cycle, and a processor with three ALUs can issue all three at once. Instruction 4 depends on 1 and 2 (it consumes R_0 and R_1), and instruction 5 depends on 4. So the dependency graph has critical-path length 3, and three cycles is the best we can do regardless of how many ALUs we have:



Definition 1.3 (Instruction-level parallelism). The *instruction-level parallelism* (ILP) of a program at a given step is the number of instructions that can be issued simultaneously without violating

dependencies. A *superscalar* processor finds independent instructions in the dynamic instruction stream and issues several of them per cycle on multiple execution units.

Empirically (Johnson, 1991, reproduced by Culler and Singh), most of the ILP in real programs is captured by a processor that can issue about four instructions per clock; building wider machines yields rapidly diminishing returns. By the late 2000s, designers had effectively saturated ILP: shipping a wider issue width no longer translated into proportionate speedup on real workloads.

1.3.3 The power wall

The other engine, clock-frequency scaling, hit a separate wall. Dynamic power dissipation in CMOS logic scales as

$$P_{\text{dyn}} \propto C \cdot V^2 \cdot f,$$

where C is the switched capacitive load, V is the core voltage, and f is the clock frequency. The maximum stable f depends on V , so pushing the clock requires raising voltage, and power grows roughly as f^3 in the relevant regime. Static (leakage) power adds a baseline that does not vanish even when transistors are idle.

The practical consequence is a thermal design budget. Apple’s M1 laptop chip sits around 13 W; an Intel Core i9 desktop CPU around 95 W; an NVIDIA RTX 4090 GPU around 450 W; a top-end supercomputer node measured in megawatts. Within any of these envelopes, you cannot simply crank the clock to recover speedups that ILP no longer delivers.

***Note: The two-wall summary.** Single-thread performance growth slowed because (1) ILP scaling tapped out and (2) frequency scaling was capped by power. Architects responded by spending the available transistors on more execution units running in parallel rather than on a single faster core. The free lunch ended.*

1.4 The Parallel Hardware Landscape

Once the design budget is spent on parallelism rather than on faster scalar cores, the hardware landscape diversifies along several axes.

1.4.1 Multi-core CPUs

A multi-core CPU is essentially several scalar cores on the same die, each with its own fetch/decode and execution units, sharing a memory hierarchy. Intel’s 10th-generation Core i9 has 10 cores; AMD’s Ryzen Threadripper 3990X has 64 cores arranged as eight chiplets of eight cores each. Programs that decompose work across cores can in principle achieve nearly P -fold speedups, subject to the same communication / load-balance / dependency constraints we saw in the demos.

1.4.2 SIMD and hyper-threading

Inside a single core, two more layers of parallelism are exposed. *SIMD* (*Single Instruction, Multiple Data*, e.g. Intel AVX) instructions operate on a vector of elements per cycle. *Hyper-threading* runs two instruction streams in the same physical core, letting the core hide latency by switching between them. Combining all three layers — multi-core, SIMD, hyper-threading — on a quad-core CPU can yield roughly 32–40× speedup over a single-threaded baseline compiled with -O3, far more than the four-fold gain from cores alone.

1.4.3 GPUs and accelerators

A modern GPU is a different point in the design space: thousands of small, simple execution units rather than a few wide, complex ones. The NVIDIA RTX 4090 contains roughly 76 billion transistors and 18,432 FP32 multipliers organized as 144 streaming multiprocessors. Data-center supercomputers (Frontier at Oak Ridge: 9,472 AMD CPUs and 37,888 Radeon GPUs, 21 MW total power) are large-scale aggregations of these parts.

1.4.4 Specialization

Efficiency, not just parallelism, is the second motivating force. The same power wall that makes parallelism necessary also makes *specialization* attractive: it is more energy-efficient to perform a fixed task on hardware designed exactly for it than to emulate that task on a general-purpose core. A modern phone SoC contains a CPU, a GPU, and a pile of fixed-function accelerators (image signal processor, video codec, neural-network engine). Data-center accelerators (Google TPU, GraphCore IPU, Cerebras Wafer-Scale Engine) push the same logic up to a different scale.

Note: *Where this course goes.* Achieving efficient processing almost always reduces to accessing data efficiently. Memory hierarchy and locality will be a recurring subject: even on a heavily parallel machine, the cost of moving data between caches, DRAM, and other processors is what determines whether a program scales.

1.5 Memory and the Cost of Data Movement

Achieving efficient processing almost always reduces to accessing data efficiently. Before we can study how parallel programs scale, we need a working mental model of memory and the cost of touching it.

1.5.1 The address space abstraction

Definition 1.4 (Address space). A computer’s memory is organised as an array of bytes. Each byte is identified by an integer *address* — its position in this array. A program’s *address space* is the set of addresses it can read and write. We assume memory is byte-addressable, so the smallest unit accessible at a given address is one byte.

A processor interacts with this array through two new instructions. A *load* reads a value from memory into a register; a *store* writes the contents of a register back to memory. Symbolically,

```
1 ld R0 ← mem[R2] ; load 4 bytes at the address held in R2 into R0
2 st mem[R2] ← R0 ; store R0 to the address held in R2
```

Everything a program “computes” is, from the processor’s perspective, a sequence of such loads, stores, and arithmetic operations on registers.

1.5.2 Latency and stalls

Definition 1.5 (Memory access latency). The *latency* of a memory access is the amount of time, measured in clock cycles or nanoseconds, between when the processor requests data and when that data is delivered.

A processor *stalls* when it cannot make progress because the next instruction depends on a previous instruction that has not yet completed. Memory accesses are a major source of stalls because their latency is large compared to the time to execute an arithmetic instruction. For the sequence

```
1 ld r0 mem[r2]
2 ld r1 mem[r3]
3 add r0, r0, r1
```

the add cannot issue until both loads have returned, and a load to main memory typically costs hundreds of cycles. The processor sits idle the whole time.

Note: Why this matters for parallel code. A program that is otherwise embarrassingly parallel will fail to scale if its threads spend most of their cycles waiting for memory. Every technique we study later — multi-threading, prefetching, locality optimisation, message passing — exists to attack some flavour of this problem.

1.5.3 Caches

The standard hardware response to high memory latency is the *cache*.

Definition 1.6 (Cache). A *cache* is on-chip storage that holds a copy of a subset of memory. If the address the processor needs is in the cache, the data can be delivered in a few cycles rather than the hundreds required to access DRAM. A cache is purely a performance feature: it does not change what a program computes, only how quickly it computes it.

Caches transfer data at the granularity of fixed-size *cache lines* (typically 64 bytes on x86), not individual bytes. The unit of bookkeeping is the line, and pulling one byte into the cache pulls its entire enclosing line. This single design choice has a large payoff because real programs exhibit two forms of *locality* that the line granularity exploits.

Definition 1.7 (Temporal and spatial locality). A reference stream has *temporal locality* if recently accessed addresses are likely to be accessed again soon, and *spatial locality* if addresses near a recently accessed address are likely to be accessed soon.

Spatial locality makes line-granularity transfer cheap on a per-byte basis: the cost of fetching one byte already paid for the next 63. Temporal locality makes the cache profitable to maintain at all: re-reads of a hot location stay on-chip.

Example: Sequential scan of an array. Suppose the cache holds two lines of 4 bytes each, with a least-recently-used (LRU) replacement policy, and a program reads addresses $0x0, 0x1, 0x2, \dots, 0xF$ in order. Address $0x0$ is a *cold miss* and pulls the line $[0x0, 0x3]$ into the cache; the next three accesses are hits (spatial locality). At $0x4$ another cold miss pulls $[0x4, 0x7]$, and so on. Each 4-byte line costs one miss and three hits, so the hit rate on the 16-byte scan is $\frac{12}{16} = 75\%$.

Example: Capacity miss. With the same 2-line cache, suppose the program now reads the entire 16-byte array and then re-reads address $0x0$. The four scan misses have evicted the original line: $0x0$'s line was kicked out when $0x8$'s line was loaded. The re-read of $0x0$ is therefore a *capacity miss* — the data was once in the cache but did not fit alongside the rest of the working set. Cold misses are unavoidable for the first touch; capacity misses are programmer-addressable through better blocking.

1.5.4 The memory hierarchy

Caches are not a single tier. A modern CPU exposes a hierarchy of caches of increasing size and increasing latency, backed by DRAM:

- *L1 cache* — on the order of 32 KB per core, ~ 4 cycles.
- *L2 cache* — on the order of 256 KB per core, ~ 12 cycles.
- *L3 cache* — on the order of 20 MB, shared across cores, ~ 38 cycles.

- DRAM — tens of GB, ~ 250 cycles.

Numbers above are representative of an Intel Kaby Lake CPU at 4 GHz; the exact figures vary by chip, but the orders of magnitude do not. A single load that the compiler emits as one line of assembly may resolve in 4 cycles or 250 cycles depending on *where* the data sits at the moment of the access, a factor of $60\times$ difference invisible to the source code.

***Note:** Why the hierarchy has the shape it does. Faster memory is smaller because the physics of fast access (short wires, many ports, low capacitance) is incompatible with the physics of dense storage. The hierarchy is the engineering compromise: small fast storage near the cores, large slow storage farther away, with the cache subsystem trying to keep the right subset close. The programmer's job is to write code whose access pattern makes that subsystem's prediction correct.*

1.6 Summary

Single-thread-of-control performance is improving very slowly. To run programs significantly faster, those programs must use multiple processing elements or specialised hardware — which means the programmer needs to reason about parallel execution rather than rely on the next generation of silicon. Writing parallel programs is non-trivial: it requires decomposing the work, balancing the load, and managing communication and synchronisation, and doing all of that well requires knowing what the machine looks like and, in particular, how data moves through it. The payoff is large: modern hardware has more processing capability than naïve sequential code reveals, and the rest of the course is about how to actually use it.